

Relatório de Laboratório

Laboratório 02 — Assistentes de IA vs. Codificação Manual: um Experimento Controlado

Curso	Engenharia de Software
Disciplina	Laboratório de Experimentação de Software
Turno / Período	Noite / 6º
Professor(a)	Danilo Maia
Laboratório	Lab02 — Assistentes de IA vs. codificação manual: um experimento controlado
Grupo (trio)	Arthur Henrique Teixeira · João Pedro Tavares · Bernardo Augusto
Link do repositório / GitHub Projects	https://github.com/BernardoApl/metrica-software
Data de entrega	23/09/2026

1. Introdução

Ferramentas de IA generativa (GitHub Copilot, ChatGPT, Claude, Gemini) tornaram-se onipresentes no desenvolvimento de software, mas ainda há pouca evidência controlada e reproduzível sobre seu real impacto em produtividade e qualidade de código — a maior parte do que se ouve é relato anedótico. Este laboratório busca preencher parte dessa lacuna com um experimento controlado, comparando a resolução de katas de programação com e sem assistente de IA, no próprio grupo (desenho crossover/within-subject), sob condições de tempo limitado (time-box).

As Questões de Pesquisa do enunciado são:

- **RQ1:** o uso de assistente de IA reduz o tempo necessário para resolver uma tarefa de programação?
- **RQ2:** o uso de assistente de IA reduz a quantidade de defeitos (testes que falham) no código produzido?
- **RQ3:** o uso de assistente de IA altera a complexidade ciclomática ou a duplicação do código produzido?

Hipóteses informais do grupo, antes de olhar os dados (formalizadas na RQ33 como H0/H1 para o teste de Wilcoxon):

- RQ1: esperávamos tempo menor com IA (o assistente reduz o tempo de digitação e de busca por sintaxe/API).
- RQ2: esperávamos taxa de sucesso maior com IA (menos erros de sintaxe e de lógica básica).
- RQ3: esperávamos complexidade ciclomática e duplicação diferentes com IA — sem uma direção clara a priori, já que IA pode tanto gerar código mais estruturado quanto mais verboso/repetitivo.

Além do enunciado, o grupo propõe três contribuições adicionais (30% de inovação, detalhadas na Seção 3.6): (a) uma auditoria automática (RQ50) que cruza os trials reais com o plano de Issues do GitHub,

identificando exatamente quais faltam ou têm rótulo divergente; (b) uma validação de integridade/rastreabilidade ponta a ponta dos trials (RQ43); (c) uma comparação descritiva complementar às RQs — mediana/IQR de complexidade, LOC e duplicação agregada por kata (RQ44) e por tratamento (RQ54).

2. Contexto

Este é o Lab02 da disciplina, conduzido após o Lab01 (mineração de repositórios do GitHub). Diferente do Lab01, cujo objeto de estudo eram repositórios públicos de terceiros, aqui o objeto de estudo é o próprio processo de trabalho dos três integrantes do grupo: cada um resolve katas de programação autorais, metade com assistente de IA habilitado e metade sem, sob um time-box de 35 minutos por tentativa (trial).

A base teórica do desenho experimental é o método GQM — Goal, Question, Metric (BASILI; CALDIERA; ROMBACH, 1994), usado para derivar as métricas de RQ1–RQ3 a partir do objetivo geral do laboratório (ver RQ33 — Hipóteses, Variáveis e Ameaças à Validade). A linguagem de implementação dos katas é Python, e a ferramenta de métricas estáticas é o Radon, escolhida porque a ferramenta indicada no enunciado para essa métrica (CK) exige Java.

3. Metodologia

3.1 Principais Desafios

- **Padronizar katas de dificuldade equivalente evitando memorização pela IA.** Katas muito conhecidos (LeetCode/HackerRank) correm o risco de a IA reproduzir uma solução já vista no treinamento em vez de efetivamente ajudar. O grupo mitigou isso escrevendo 6 katas autorais com contexto de negócio fictício (RQ32), reduzidos a 4 na execução prática (ver nota abaixo).
- **Redução do escopo de 6 para 4 katas.** A RQ33 planejou originalmente 6 katas $\times$ 3 integrantes = 18 trials. Na execução (Sprint 2), o grupo convergiu para 4 katas (frete progressivo, cálculo de estacionamento, deduplicador de contatos, tarifa de energia) $\times$ 3 integrantes = 12 trials, sem atualizar formalmente a documentação da RQ32/RQ33 — um ponto a corrigir em iterações futuras.
- **Arquivo `solucao.py` compartilhado por kata.** Cada kata tem um único arquivo de solução no repositório, usado por todos os integrantes. Isso causou sobrescritas entre trials de pessoas diferentes e, em pelo menos dois casos (kata02 e kata04 do bblp, tratamento `sem_ia`), trials com duração próxima de zero porque o `solucao.py` já continha uma solução completa de uma sessão anterior — o cronômetro mediu apenas o tempo de rodar os testes, não de escrever a solução.
- **Erros de rotulagem manual.** Foram encontrados: uma Issue usada por engano (kata03 do Arthur registrada sob a Issue de outro integrante), dois trials do João com o tratamento (`com_ia/sem_ia`) trocado em relação ao plano, e uma linha duplicada no CSV de tempos (mesmo `trial_id` registrado duas vezes com nomes de kata diferentes).
- **Dependência ausente.** O `scipy`, necessário para o teste de Wilcoxon (RQ51–53) calcular p-valor, não estava listado no `requirements.txt` — os scripts rodavam silenciosamente apenas com estatística descritiva, sem o teste de hipótese, até ser detectado na revisão da Sprint 3.
- **Duplicação de código não mensurável no ambiente de coleta.** A métrica de duplicação (`jscpd`) depende de Node.js/npx, indisponível nas máquinas usadas para os trials — nenhum trial teve duplicação medida.

3.2 Tomadas de Decisão

- **Assistente de IA:** o desenho original (RQ33) definiu o ChatGPT (versão gratuita) como assistente único do grupo, para manter o tratamento com_ia comparável entre integrantes. Na prática, o Bernardo e o João usaram ChatGPT; o Arthur usou o Claude (via Claude Code) em seus trials com_ia. Essa divergência não foi corrigida a tempo deste relatório e é tratada como ameaça à validade adicional na Seção 4.3.
- **Definição operacional de RQ1 (tempo):** time-to-green (tempo até o comando de aceitação retornar código 0), com censura no time-box de 35 min (2100 s) em vez de descarte — trial que estoura o tempo entra na análise como censurado, não é removido.
- **Definição operacional de RQ2 (defeitos):** taxa de sucesso (trial termina com todos os testes passando dentro do time-box), preferida a contagem bruta de falhas por normalizar katas com números diferentes de testes.
- **Definição operacional de RQ3:** complexidade ciclomática média normalizada por LOC (não em valor absoluto), para controlar o efeito de código gerado por IA ser potencialmente mais verboso.
- **Linguagem e ferramental:** Python foi escolhido em função da disponibilidade e simplicidade do Radon para métricas estáticas (CK, sugerido no enunciado, exige Java).
- **Estatística descritiva:** mediana e IQR em vez de média e desvio-padrão nas tabelas, e teste de Wilcoxon pareado (não paramétrico) na análise inferencial — decisão justificada pelo N pequeno do desenho within-subject (RQ33).
- **Limite de WIP:** a coluna "Doing" do board (GitHub Projects) foi configurada com limite de 3 itens simultâneos, para evitar trabalho em paralelo excessivo e reduzir troca de contexto entre os três integrantes.

3.3 Etapas

Divisão de responsabilidades por sprint, refletindo os autores reais dos commits/Issues no repositório:

Sprint	Entregas	Responsável(is)	Issues
Lab02S01	Script de cronometragem (RQ28); ambiente e script de métricas estáticas (RQ30) e sua validação (RQ29); seleção/validação de katas (RQ32) e hipóteses/variáveis/ameaças à validade (RQ33)	Bernardo (RQ28); Arthur (RQ29, RQ30); João (RQ32, RQ33)	#28–#30, #32, #33
Lab02S02	Execução dos trials: cada integrante resolveu as 4 katas, metade com IA e metade sem IA, em ordem contrabalanceada entre os três	Arthur (#37,38,39,48); Bernardo (#34,35,36,47); João (#40,41,42,49)	#34–#42, #47–#49
Lab02S03	Auditoria de cobertura (RQ50); testes de Wilcoxon para RQ1/RQ2/RQ3 (RQ51–53); pipeline de dataset unificado (RQ57); validação das métricas estáticas (RQ31); dashboard (RQ58); consolidação por tratamento (RQ54); correções de dados e revisão final do relatório	João (RQ50, RQ51–53, RQ57); Bernardo (RQ31, RQ58, 1ª versão do relatório); Arthur (RQ54, correções de dados/dependências,	#31, #50–#54, #57, #58

Sprint	Entregas	Responsável(is)	Issues
		revisão final do relatório)	

Configuração do processo: board no GitHub Projects (v2) com colunas Backlog → To Do → Doing → Review → Done, limite de WIP de **3** itens na coluna Doing. O grupo optou por não anexar captura de tela do board nesta versão do relatório.

3.4 Ferramentas

- **Python 3** com `unittest` — implementação dos katas e testes de aceitação.
- **Radon** (`cc`, `raw`, `mi`) — complexidade ciclomática, LOC/SLOC/LLOC e índice de manutenibilidade.
- **jscpd** — planejado para medir duplicação de código; não executado nos trials reais por ausência de Node.js/npx no ambiente de coleta (ver 3.1).
- **Pandas** — consolidação do dataset unificado (RQ57), unindo tempo (RQ28) e métricas estáticas (RQ30) por `trial_id`.
- **SciPy** — teste de Wilcoxon pareado (RQ51, RQ52, RQ53).
- **Matplotlib** — dashboard (RQ58) e gráficos de comparação (RQ44, RQ54).
- **ChatGPT (versão gratuita)** — assistente de IA usado pelo Bernardo e pelo João nos trials `com_ia`.
Claude / Claude Code — assistente usado pelo Arthur em seus trials `com_ia`, divergindo do padrão combinado pelo grupo.
- **GitHub Projects (v2)** — ferramenta de processo do grupo. Link: <https://github.com/BernardoApl/metrica-software>
- **Scripts próprios** de cronometragem (`rq28_cronometragem.py`), coleta de métricas estáticas (`rq30_metricas_estaticas.py`), validação (`rq29`, `rq31`, `rq43`), auditoria (`rq50`), testes estatísticos (`rq51–rq53`), pipeline (`rq57`) e comparações descritivas (`rq44`, `rq54`) — todos no diretório `codigo-fonte/`.

3.5 Tabela de Métricas

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ1	Tempo até passar nos testes (time-to-green)	Tempo entre o início do trial e o comando <code>testar</code> retornar código 0 nos testes de aceitação; se o time-box (2100 s) é atingido sem sucesso, registra-se 2100 s (censurado), sem descarte	Segundos	<code>rq28_cronometragem.py</code> (script próprio)
RQ2	Taxa de sucesso	% de trials que terminam com todos os testes de aceitação passando dentro do time-box	% (0-1)	<code>rq28_cronometragem.py</code> , campo <code>sucesso</code>

RQ	Métrica	Definição Operacional	Unidade	Ferramenta / Fonte
RQ3	Complexidade ciclomática por LOC	Complexidade ciclomática média das funções do <code>solucao.py</code> final (Radon cc) dividida pelo LOC do arquivo	Adimensional	<code>rq30_metricas_estaticas.py</code> (Radon)
RQ3	Duplicação de código	% de linhas duplicadas no <code>solucao.py</code> final (jscpd, min-lines=3, min-tokens=20)	%	<code>rq30_metricas_estaticas.py</code> (jscpd) — não coletada
RQ3 (controle)	LOC	Linhas de código do <code>solucao.py</code> final	Linhas	<code>rq30_metricas_estaticas.py</code> (Radon raw)

3.6 Inovações Propostas pelo Grupo (30% da nota)

- **RQ50 — Auditoria de cobertura dos trials.** Script que cruza o CSV real da RQ28 com o plano de 12 trials esperados (participante × kata × tratamento × Issue do GitHub), classificando cada trial como OK, divergente (rótulo ou métrica faltando) ou apontando Issues nunca executadas. Foi essencial na Sprint 3 para identificar exatamente as 3 Issues faltantes (#34, #35, #40) e as 2 com tratamento trocado (#41, #49), substituindo uma checagem manual sujeita a erro.
- **RQ43 — Validação de integridade/rastreabilidade.** Cruza RQ28 e RQ30 com as evidências complementares de cada trial (diretório, `solucao.py`, registro de prompt para trials com_ia), gerando um relatório por trial que indica se ele é totalmente rastreável — usado para confirmar que os 4 trials do Arthur estavam corretos de ponta a ponta.
- **RQ44 — Comparação descritiva por kata e tratamento.** Consolida o CSV da RQ30 em mediana/IQR de complexidade, LOC e duplicação, agrupando por (kata, tratamento) — permite comparar, por exemplo, se a kata04 ficou mais complexa com IA especificamente, e não só em agregado.
- **RQ54 — Consolidação por tratamento (complementar à RQ44).** Mesma lógica da RQ44, mas agregando todos os katas de cada tratamento numa única linha — dá a visão geral e direta da RQ3 (complexidade/LOC/duplicação com_ia vs. sem_ia) usada na Seção 4.

Essas quatro contribuições aparecem nos resultados e na discussão a seguir: a RQ50 orienta diretamente os "Próximos Passos" da Seção 5, e a RQ44/RQ54 fundamentam a leitura da RQ3 na Seção 4.3.

4. Resultados

4.1 Coleta de Dados

O plano do experimento previa 12 trials (4 katas × 3 integrantes). O CSV real de tempos (RQ28) tem **14 registros** (após remover uma linha duplicada — mesmo `trial_id` gravado duas vezes com nomes de kata diferentes). Cruzando contra o plano de Issues (RQ50):

- **6 trials** batem exatamente com o plano (participante, kata, tratamento e Issue corretos);
- **3 trials** estão divergentes: 2 do João com o tratamento trocado (Issues #41 e #49) e 1 do João com sucesso mas ainda sem métrica estática vinculada (Issue #42);

- **3 Issues do plano nunca foram executadas:** #34 e #35 (Bernardo), #40 (João);
- Os **2 registros restantes** são tentativas extras do Bernardo nas katas 02 e 04 (`sem_ia`), repetidas sobre um `solucao.py` que já existia antes do trial — daí a duração próxima de zero (ver 3.1 e 4.3).

A RQ30 (métricas estáticas) tem **10 registros** vinculados a trials da RQ28 — inicialmente eram apenas 2 (só os trials `com_ia` do Bernardo); o Arthur completou a coleta para seus 4 trials e para os trials históricos do Bernardo e do João recuperados do próprio histórico do Git, elevando a cobertura para permitir o teste de Wilcoxon da RQ3 pela primeira vez com os 3 integrantes.

Nenhum trial teve a duplicação de código medida (`jscpd` indisponível no ambiente). Todos os 14 trials da RQ28 terminaram com status `sucesso` — não há trials censurados ou interrompidos no dataset atual.

4.2 Visualização Gráfica

RQ1 — o uso de assistente de IA reduz o tempo de resolução? Gráfico de pontos conectados (before/after), um por participante, tempo em escala logarítmica dado o intervalo de 1,7 s a 974 s. Arthur (1,71 s → 787,29 s) e João (72,25 s → 974,08 s) sobem de `com_ia` para `sem_ia`, no sentido esperado pela hipótese. O Bernardo desce (35,13 s → 0,28 s) — padrão invertido, causado pelo problema de dados descrito na Seção 4.3 (código já existente antes do trial `sem_ia`).

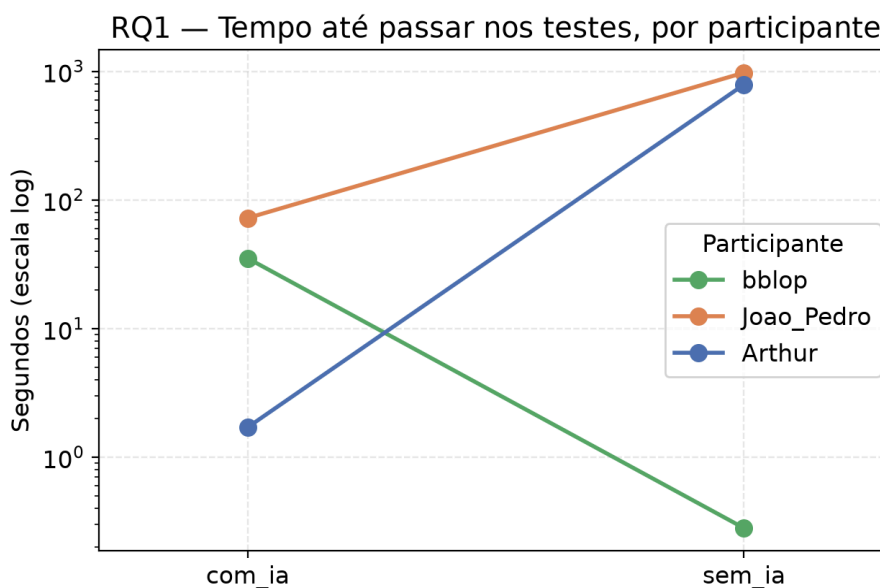


Figura 1 — RQ1: tempo até passar nos testes, `com_ia` vs. `sem_ia`, por participante (escala log).

RQ2 — o uso de assistente de IA reduz a quantidade de defeitos? Os 14 trials coletados têm taxa de sucesso de 100% nos dois tratamentos, para os três participantes — barras idênticas, sem variação para comparar. Isso reflete o desenho de coleta (só o `solucao.py` final, já passando nos testes, é commitado), não necessariamente ausência de defeitos durante a resolução.

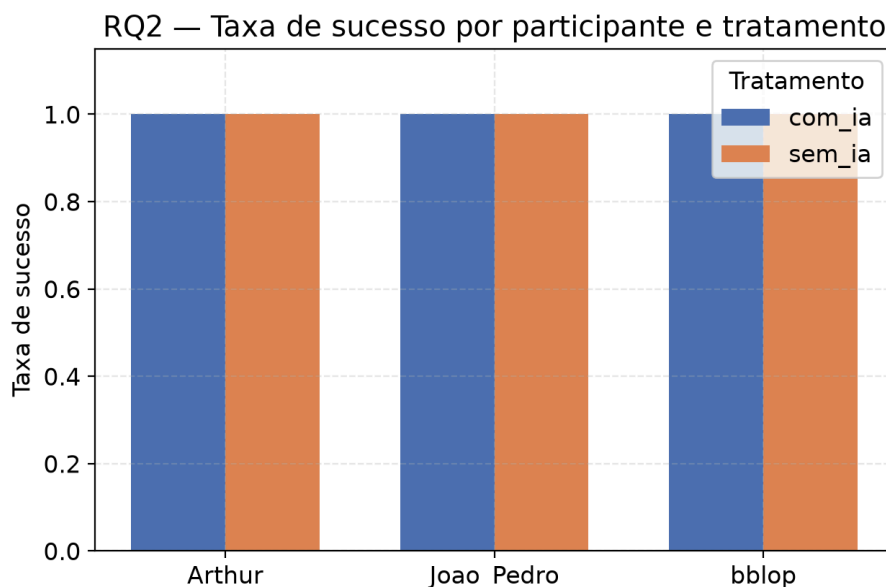


Figura 2 — RQ2: taxa de sucesso por participante e tratamento (sem variação observável).

RQ3 — o uso de assistente de IA altera a complexidade ou a duplicação do código? Mesmo formato de gráfico pareado, agora para complexidade ciclomática por LOC. Sem padrão consistente entre participantes: Arthur sobe levemente (0,163 → 0,205), João e Bernardo descem (0,194 → 0,125 e 0,304 → 0,159). As três linhas convergem para uma faixa próxima em sem_ia (0,125–0,205), sem separação clara por tratamento — consistente com o Wilcoxon não significativo ($p = 0,5000$). Duplicação não aparece por não ter sido coletada em nenhum trial.

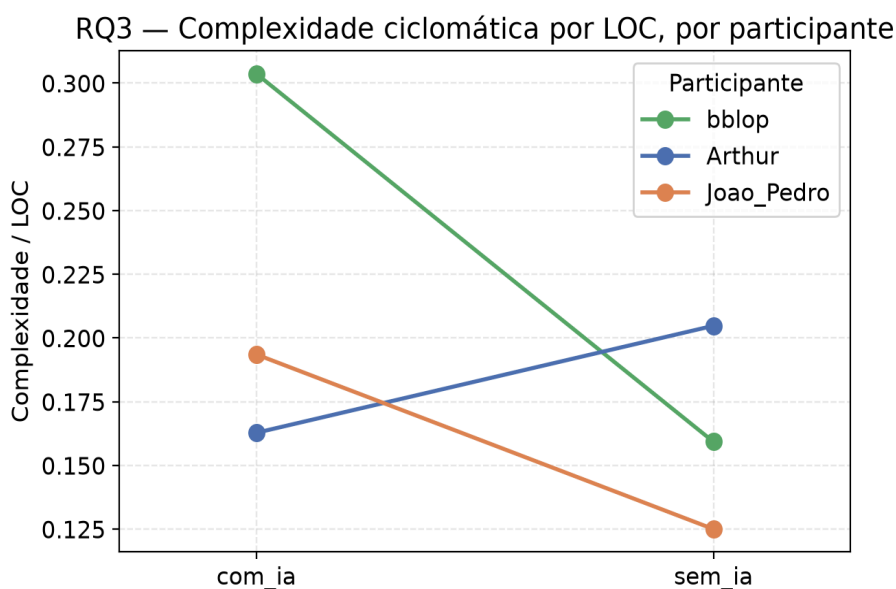


Figura 3 — RQ3: complexidade ciclomática por LOC, com_ia vs. sem_ia, por participante.

Visão geral (RQ1 + RQ3 + cobertura da coleta): o dashboard da RQ58 (Figura 4) consolida, num único painel, a contagem de trials por tratamento, o tempo mediano por participante, a cobertura de

Issues/RQ30 e a dispersão de complexidade por LOC — útil para conferir a consistência dos números acima de uma vez só.

RQ58 - Dashboard do LAB02 a partir da RQ57

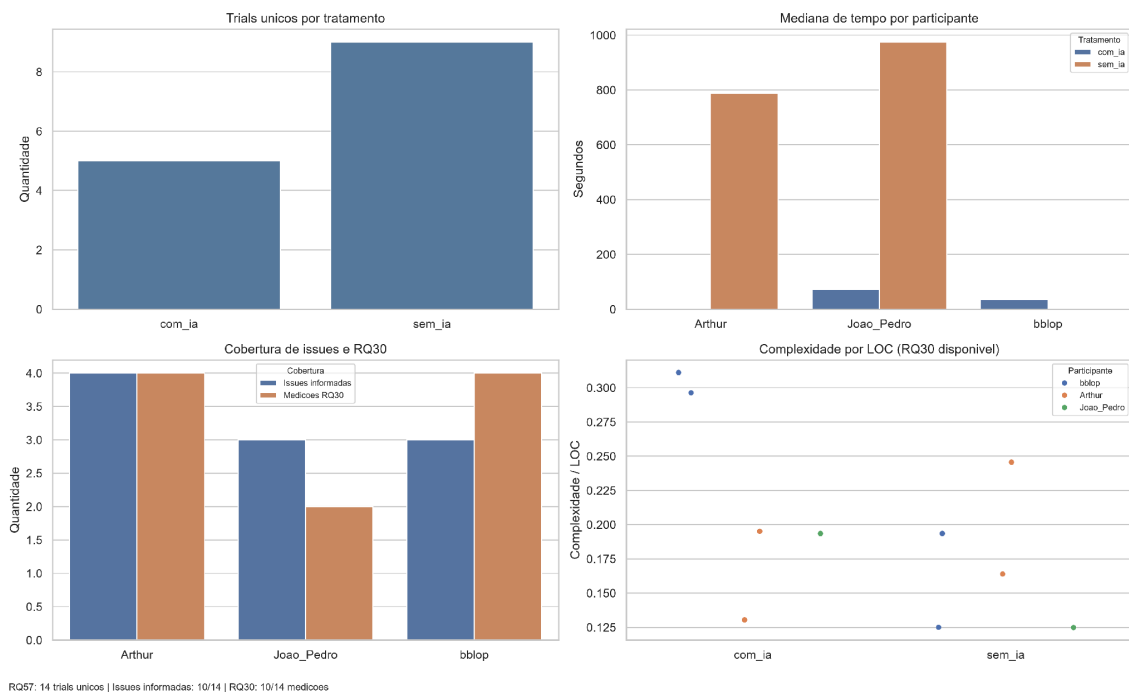


Figura 4 — Dashboard RQ58: trials por tratamento, tempo mediano por participante, cobertura de issues/RQ30 e complexidade por LOC.

4.3 Discussão

RQ1 (tempo): a hipótese informal era de tempo menor com IA. O resultado descritivo aponta nessa direção (mediana *com_ia* 35,13 s vs. *sem_ia* 787,29 s), e o teste de Wilcoxon ($W+ = 1,0$, $W- = 5,0$, $p = 0,5000$) **não rejeita H_0** — não há evidência estatística de diferença, apesar da tendência descritiva. A hipótese fica **não confirmada estatisticamente**, embora não refutada.

RQ2 (defeitos): a hipótese informal era de taxa de sucesso maior com IA. Não é possível testá-la como coletada: 100% dos 14 trials terminaram com sucesso em ambos os tratamentos, então a diferença pareada é sempre zero. Isso é consistente com o próprio desenho de coleta — apenas o `solucao.py` final (já passando nos testes) é commitado, então trials malsucedidos/censurados não aparecem no CSV. A hipótese fica **sem resposta** com os dados atuais.

RQ3 (estrutura do código): a hipótese informal não tinha direção definida. O teste de Wilcoxon sobre complexidade por LOC ($p = 0,5000$) **não rejeita H_0** ; LOC mediano de controle é próximo entre tratamentos (32,0 vs. 27,5), sem indício de verbosidade sistematicamente maior num deles. A métrica de duplicação, escolhida na RQ33 como complementar, **não foi coletada** — a RQ3 fica parcialmente respondida.

Ameaças à validade observadas na execução (complementando as já previstas na RQ33):

1. **N extremamente pequeno.** Com 3 pares por RQ, o menor p-valor possível no teste de Wilcoxon já é 0,25 — a ausência de significância estatística era praticamente esperada antes mesmo de rodar o teste. Os resultados são preliminares.
2. **Mistura de assistentes de IA.** O desenho da RQ33 previa um único assistente (ChatGPT) para manter o tratamento `com_ia` comparável; o Arthur usou o Claude, o que introduz uma variável de confusão não controlada entre participantes.
3. **Qualidade heterogênea dos dados de tempo.** Os trials `sem_ia` do Bernardo nas katas 02/04 têm mediana de 0,28 s — incompatível com escrever uma solução do zero (o enunciado estima 20-25 min). O histórico do Git confirma que o código já existia antes desses trials, então essas medições refletem "tempo para rodar os testes", não "tempo de resolução sem IA". Isso distorce diretamente a RQ1: o par do Bernardo é o único dos três em que `sem_ia` aparece mais rápido que `com_ia`, na direção oposta da hipótese e dos outros dois participantes.
4. **Efeito de aprendizado e memorização.** Mitigado pelo uso de katas autorais (RQ32), mas o efeito de aprendizado entre katas dentro de um mesmo trial de 35 min não foi medido diretamente.

Contribuição das inovações (3.6): a RQ50 foi o que permitiu identificar, com precisão, que só 6 dos 12 trials batiam integralmente com o plano — sem ela, a divergência dos rótulos do João e a ausência das Issues #34/#35/#40 provavelmente passariam despercebidas até a entrega final. A RQ44/RQ54 mostram que, mesmo agregando por kata ou só por tratamento, as medianas de complexidade e LOC permanecem próximas entre `com_ia` e `sem_ia` — reforçando, de forma descritiva, a mesma conclusão de "sem diferença detectável" que o Wilcoxon da RQ3 já indicava, e não revelando nenhum padrão por kata que o agregado escondesse.

5. Conclusão

Com os dados coletados até o momento, nenhuma das três Questões de Pesquisa apresentou diferença estatisticamente significativa entre os tratamentos `com_ia` e `sem_ia`: RQ1 (tempo) e RQ3 (complexidade por LOC) tiveram $p = 0,5000$ no teste de Wilcoxon pareado, e a RQ2 (taxa de sucesso) não pôde ser testada por falta de variância nos dados coletados. Esses resultados devem ser lidos como **preliminares**, não conclusivos — o principal fator limitante é o tamanho amostral (apenas 3 pares completos por RQ), seguido pela cobertura incompleta da coleta (3 das 12 Issues planejadas nunca foram executadas) e pela ausência total de dados de duplicação de código.

As principais limitações do estudo, em ordem de impacto, são: (1) N pequeno, que restringe qualquer teste de hipótese a um poder estatístico muito baixo; (2) qualidade heterogênea de parte dos dados de tempo, especificamente os trials `sem_ia` do Bernardo, cujo `solucao.py` já existia antes da execução do cronômetro; (3) mistura de assistentes de IA (ChatGPT e Claude) entre os integrantes, não prevista no desenho original; (4) métrica de duplicação nunca coletada, por indisponibilidade de `Node.js/npx` no ambiente.

Com mais tempo, o grupo faria diferente: (a) isolaria uma cópia de `solucao.py` por trial em vez de um arquivo único compartilhado por kata, eliminando o risco de sobrescrita e de medições sobre código pré-existente; (b) fixaria um único assistente de IA testado no ambiente de todos os integrantes antes do início da coleta; (c) validaria a disponibilidade de todas as dependências (incluindo `scipy` e `Node.js/jscpd`) antes da Sprint 2, não durante a análise da Sprint 3. Das inovações propostas (30%), a que mais valeria a pena expandir é a **RQ50 (auditoria automática de cobertura)** — rodá-la continuamente durante a coleta, não só ao final, teria evitado a maior parte dos problemas de rotulagem descritos neste relatório.

6. Referências

ZUSE, Horst. A framework of software measurement. Walter de Gruyter, 2013.

BASILI, V. R.; CALDIERA, G.; ROMBACH, H. D. The Goal Question Metric Approach. In: Encyclopedia of Software Engineering. Wiley, 1994.